The example shows the benefit of Ruby's approach to iterators. The Array class does what it does best, accessing array elements, and leaves the application code to concentrate on its particular requirement (in this case, finding an entry that meets some criteria).

Another common iterator is map (also sometimes known as collect), which takes each element from the collection and passes it to the block. The results returned by the block are used to construct a new array. The following example uses the String#succ method, which increments a string value:

```
["H", "A", "L"].map { |x| x.succ } # => ["I", "B", "M"]
```

The implementation of map looks something like this:

```
class Array
  def map
    result = []
    each do |value|
      result << yield(value)
    end
    result
  end
end
```

We start off with an empty result array, and for each element in the array, yield is invoked on the block and the resulting value is appended to the array. At the end of the method the result, now containing all the individual values that have been returned by blocks, is itself returned.

Iterators are not limited to accessing existing data in arrays and hashes. As we saw in the Fibonacci example, an iterator can return derived values. This capability is used by Ruby's input and output classes, which implement an iterator interface that returns successive lines (or bytes) in an I/O stream. In other words, they implement an each method that invokes its block once for each line in the file, so you can iterate through a file like so:

```
f = File.open("testfile")
f.each do |line|
  puts "The line is: #{line}"
end
f.close
```

produces:

```
The line is: This is line one
The line is: This is line two
The line is: This is line three
The line is: And so on...
```

Sometimes you want to keep track of how many times you've been through the block. The with_index method is your friend. It is added as an additional method call after an iterator, and adds a sequence number to each value returned by that iterator. The original value and that sequence number are then passed to the block:

```
f = File.open("testfile")
f.each.with_index do |line, index|
  puts "Line #{index} is: #{line}"
end
f.close
```